<< IERG 3810 >>

<< Microprocessor System Design Laboratory >>

Report on Experiment <<No.1>>

<< General Purpose Inputs/Outputs (GPIO) >>

Group: BL 01
Members: Yeung Man, Thomas (1155203181)
Wong Man Hei, Jimmy (1155213238)

Submission Date: 28/09/2025

Disclaimer
I declare that the assignment here submitted is original except for source material explicitly acknowledged, and that the same or related material has not been previously submitted for another course. I also acknowledge that I am aware of University policy and regulations on honesty in academic work, and of the disciplinary guidelines and procedures applicable to breaches of such policy and regulations, as contained in the website http://www.cuhk.edu.hk/policy/academichonesty/

Signature	Name	Date
	Yeung Man, Thomas	**28/09/2025**
Signature	Name	Date
	Wong Man Hei, Jimmy	**28/09/2025**

I. OBJECTIVES

1. To explore the configuration of General Purpose Input/Output (GPIO) pins by setting them as both input and output, enabling interaction with external components.

2. To gain experience in developing embedded applications using the C programming language, specifically utilizing the Standard Firmware Library to configure hardware registers.

3. To design and implement custom libraries tailored to the project board, promoting modularity, reusability, and better organization of code.

II. DATA ANALYSIS

<Procedures and measured/captured data/graph>

Experiment 1.1: Set a GPIO as an output and drive a LED with standard peripheral library.

1. Step-by-Step Procedures Enter the code shown in Figures 1-6 into the main.c file of your project. Include the source files stm32f10x_rcc.c and stm32f10x_gpio.c in the Fw_lib folder of project.
2. Compile the program and download the binary output to the project board.
3. Modify the Delay function so that LED DS0 flashes with a one-second interval (one second ON, one second OFF).

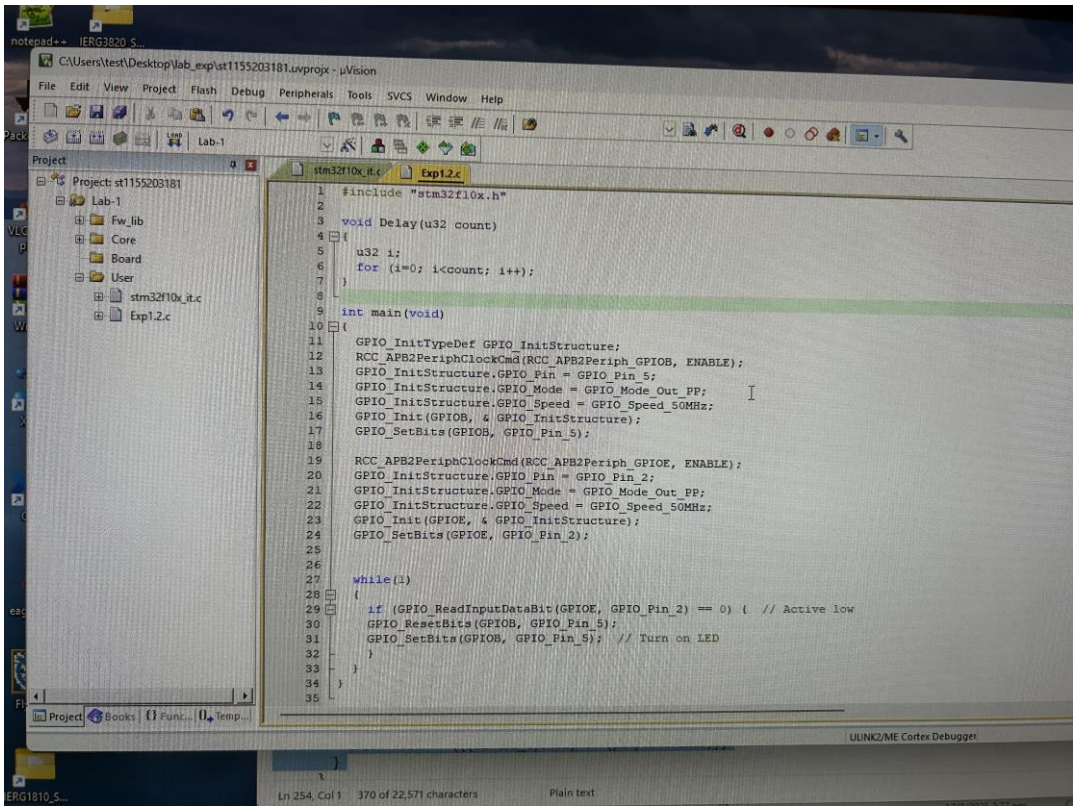
```
1  #include "stm32f10x.h"
2
3  void Delay(u32 count)
4    {
5    u32 i;
6    for (i=0; i<count; i++);
7    }
8
9  int main(void)
10 {
11   GPIO_InitTypeDef GPIO_InitStructure;
12   RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);
13   GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5;
14   GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
15   GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
16   GPIO_Init(GPIOB, & GPIO_InitStructure);
17   GPIO_SetBits(GPIOB, GPIO_Pin_5);
18   while(1)
19   {
20     GPIO_ResetBits(GPIOB, GPIO_Pin_5);
21     Delay(1000000);
22     GPIO_SetBits(GPIOB, GPIO_Pin_5);
23     Delay(1000000);
24   }
25 }
26
```

(Figure 1)

- After downloading the program, LED DS0 successfully flashed at one-second intervals.
- The delay function worked as expected, demonstrating correct timing control.
- GPIO configuration was verified through the LED’s response to the programmed logic.

Experiment 1.2: Read a key from GPIO input and drive a LED with standard peripheral library.

1. Modify the original code from Figures 1 to 6 in the main.c file.
2. Add the pin setup procedure for KEY2, configuring it as an input with an internal pull-up resistor.
3. If KEY2 is pressed (logic LOW), turn LED DS0 ON.
4. If KEY2 is not pressed, turn LED DS0 OFF.
5. Compile the program and download the binary to the project board.
6. Test the setup by pressing KEY2 and observing the LED behavior.

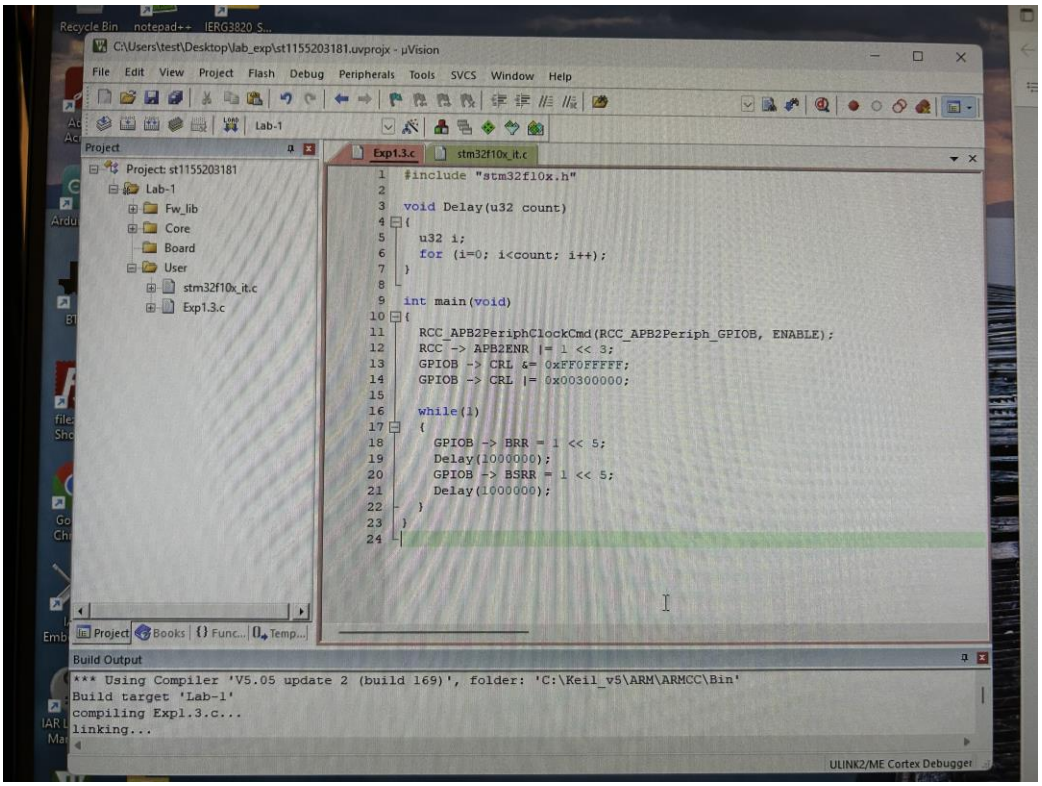


(Figure 2)

- KEY2 was successfully configured as an input with pull-up.
- When KEY2 was pressed, LED DS0 turned ON as expected.
- When KEY2 was released, LED DS0 turned OFF.
- The system responded correctly to user input, confirming proper GPIO input handling.

Experiment 1.3: Set a GPIO as an output and drive an LED with register setting

1. Modify the original code from Figures 1 to 6 in the main.c file.
2. Replace the GPIO configuration lines with register-level settings. Understand and describe the purpose of each line in your report.
3. Configure GPIOB pin-5 using both the Standard Firmware Library and direct register access
4. Modify the LED DS0 control logic to use both GPIO_SetBits() / GPIO_ResetBits() and direct register access (BSRR / BRR).
5. Compile the program and download it to the board.
6. Observe the LED behavior to confirm correct configuration and control.



(Figure 3)

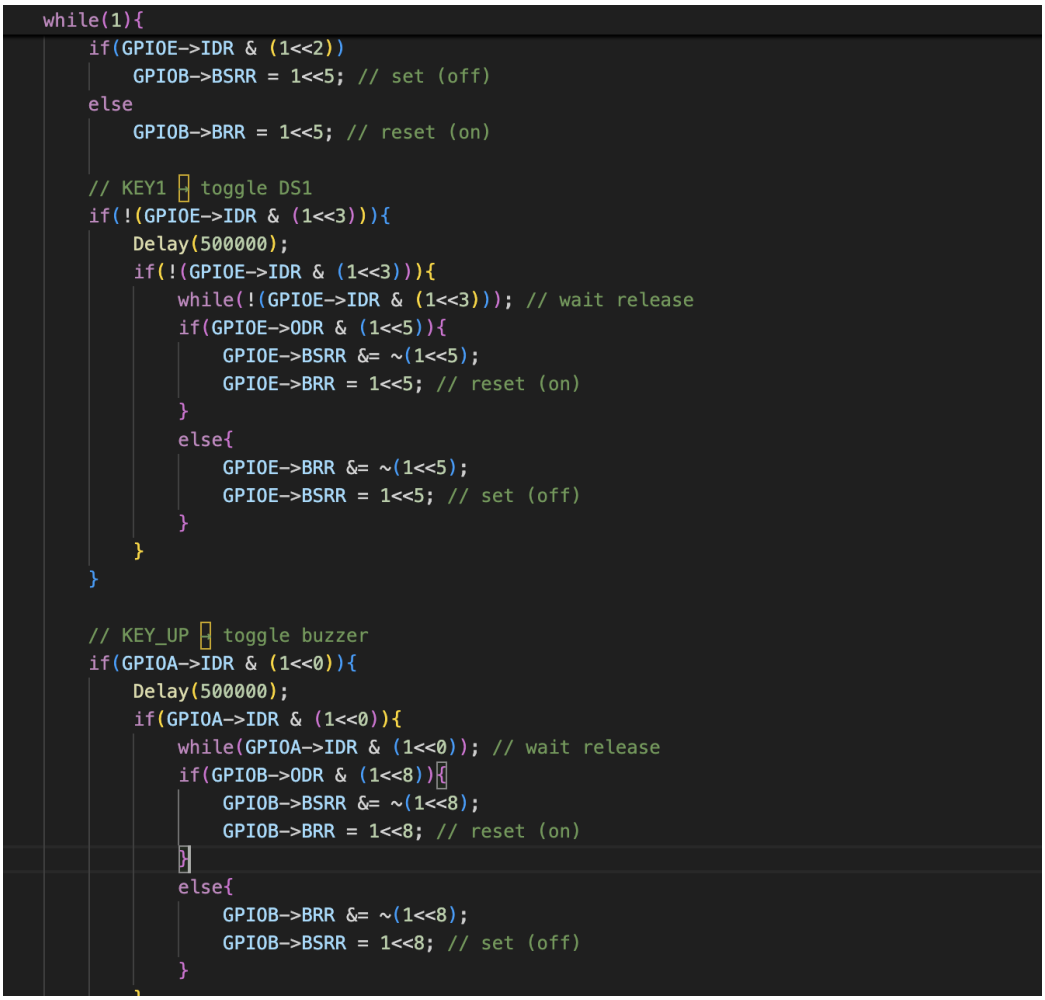
- GPIOB pin-5 was successfully configured using both library and register-level methods.
- LED DS0 responded correctly to control commands, turning ON and OFF as expected.
- Understanding register-level configuration provided deeper insight into how the microcontroller handles GPIO settings.

Experiment 1.4: Read a key from GPIO input and drive an LED with register setting.

1. The program from Experiment 1.3 was modified to use direct register settings for GPIO configuration and control.
2. The following GPIO pins were configured:
    a. PB5 (LED DS0) as output push-pull.
    b. PE5 (LED DS1) as output push-pull.
    c. PB8 (Buzzer) as output push-pull.
    d. PE2 (KEY2) as input with internal pull-up.
    e. PE3 (KEY1) as input with internal pull-up.
    f. PA0 (KEY_UP) as input with pull-down.
3. The program logic was implemented as follows:
    a. KEY2: When pressed, LED DS0 turns ON; otherwise, it remains OFF.
    b. KEY1: When pressed, LED DS1 toggles between ON and OFF.
    c. KEY_UP: When pressed, the buzzer toggles between ON and OFF.
4. Debounce logic was added using a delay loop to ensure stable button press detection and prevent multiple toggles from a single press.

```
int main(void){
    // Enable clocks
    RCC->APB2ENR |= 1<<3; // GPIOB
    RCC->APB2ENR |= 1<<6; // GPIOE
    RCC->APB2ENR |= 1<<2; // GPIOA

    // DS0 (PB5)
    GPIOB->CRL &= 0xFF0FFFFF;
    GPIOB->CRL |= 0x00300000;
    GPIOB->BSRR = 1<<5; // set (off)

    // DS1 (PE5)
    GPIOE->CRL &= 0xFF0FFFFF;
    GPIOE->CRL |= 0x00300000;
    GPIOE->BSRR = 1<<5; // set (off)

    // Buzzer (PB8)
    GPIOB->CRH &= 0xFFFFFFF0;
    GPIOB->CRH |= 0x00000003;
    GPIOB->BRR = 1<<8; // reset (on)

    // KEY2 (PE2) input with pull-up
    GPIOE->CRL &= 0xFFFFF0FF;
    GPIOE->CRL |= 0x00000800;
    GPIOE->BSRR |= 1<<2;

    // KEY1 (PE3) input with pull-up
    GPIOE->CRL &= 0xFFFF0FFF;
    GPIOE->CRL |= 0x00008000;
    GPIOE->BSRR |= 1<<3;

    // KEY_UP (PA0) input with pull-down
    GPIOA->CRL &= 0xFFFFFFF0;
    GPIOA->CRL |= 0x00000008;
    GPIOA->BRR |= 1<<0;
```

(Figure 4)



```
while(1){
    if(GPIOE->IDR & (1<<2))
        GPIOB->BSRR = 1<<5; // set (off)
    else
        GPIOB->BRR = 1<<5; // reset (on)

    // KEY1 - toggle DS1
    if(!(GPIOE->IDR & (1<<3))){
        Delay(500000);
        if(!(GPIOE->IDR & (1<<3))){
            while(!(GPIOE->IDR & (1<<3))); // wait release
            if(GPIOE->ODR & (1<<5)){
                GPIOE->BSRR &= ~(1<<5);
                GPIOE->BRR = 1<<5; // reset (on)
            }
            else{
                GPIOE->BRR &= ~(1<<5);
                GPIOE->BSRR = 1<<5; // set (off)
            }
        }
    }

    // KEY_UP - toggle buzzer
    if(GPIOA->IDR & (1<<0)){
        Delay(500000);
        if(GPIOA->IDR & (1<<0)){
            while(GPIOA->IDR & (1<<0)); // wait release
            if(GPIOB->ODR & (1<<8)){
                GPIOB->BSRR &= ~(1<<8);
                GPIOB->BRR = 1<<8; // reset (on)
            }
            else{
                GPIOB->BRR &= ~(1<<8);
                GPIOB->BSRR = 1<<8; // set (off)
            }
```

(Figure 5)

- KEY2 successfully controlled LED DS0. The LED turned ON when the button was pressed and OFF when released.
- KEY1 correctly toggled LED DS1. Each press changed the LED state from ON to OFF or vice versa.
- KEY_UP toggled the buzzer ON and OFF as expected. The buzzer responded reliably to each button press.
- All GPIO configurations and control logic worked as intended using register-level programming.
- The debounce mechanism effectively prevented unintended multiple toggles.

Experiment 1.5: Create your own library for the project board.

1. The program from Experiment 1.4 was modified to improve code modularity and maintainability by separating hardware initialization into sub-routines.

2. Three initialization functions were created:

    a. IERG3810_KEY_Init() – for configuring KEY2, KEY1, and KEY_UP.

    b. IERG3810_Buzzer_Init() – for configuring the buzzer (PB8).

    c. IERG3810_LED_Init() – for configuring LED DS0 (PB5) and DS1 (PE5).

3. These functions were implemented in separate source files:

    a. IERG3810_KEY.c

    b. IERG3810_Buzzer.c

    c. IERG3810_LED.c

4. Corresponding header files were created to declare the functions:

    a. IERG3810_KEY.h

    b. IERG3810_Buzzer.h

    c. IERG3810_LED.h All .c and .h files were stored in a folder named “Board” to organize board-specific code.

5. The new source files were added to the project using the Manage Project Items interface (as shown in Figure 1-15).

6. The main.c file was updated to call the initialization functions instead of directly configuring the registers (as shown in Figure 1-16).

7. The program was compiled and downloaded to the development board for testing.

8. Additional sub-routines may be created in the future to further modularize LED, key, and buzzer control logic.

(Figure 6)

```c
#include "stm32f10x.h"
#include "IERG3810_KEY.h"
#include "IERG3810_LED.h"
#include "IERG3810_Buzzer.h"

// Delay function
void Delay(u32 count) {
    u32 i;
    for (i = 0; i < count; i++);
}

int main(void) {
    // Initialization
    IERG3810_KEY_Init();
    IERG3810_LED_Init();
    IERG3810_Buzzer_Init();

    while (1) {
        // KEY2 → DS0 control
        if (GPIOE->IDR & (1 << 2)) {
            GPIOB->BSRR = 1 << 5; // Set (LED OFF)
        } else {
            GPIOB->BRR = 1 << 5; // Reset (LED ON)
        }
```

(Figure 7)

```c
int main(void) {
    while (1) {
        // KEY1 → toggle DS1
        if (!(GPIOE->IDR & (1 << 3))) {
            Delay(1000000);
            if (!(GPIOE->IDR & (1 << 3))) {
                while (!(GPIOE->IDR & (1 << 3))); // Wait for release

                if (GPIOE->ODR & (1 << 5)) {
                    GPIOE->BSRR &= ~(1 << 5);
                    GPIOE->BRR = 1 << 5; // Reset (LED ON)
                } else {
                    GPIOE->BRR &= ~(1 << 5);
                    GPIOE->BSRR = 1 << 5; // Set (LED OFF)
                }
            }
        }

        // KEY_UP → toggle buzzer
        if (GPIOA->IDR & (1 << 0)) {
            Delay(1000000);
            if (GPIOA->IDR & (1 << 0)) {
                while (GPIOA->IDR & (1 << 0)); // Wait for release

                if (GPIOB->ODR & (1 << 8)) {
                    GPIOB->BSRR &= ~(1 << 8);
                    GPIOB->BRR = 1 << 8; // Reset (Buzzer ON)
                } else {
                    GPIOB->BRR &= ~(1 << 8);
                    GPIOB->BSRR = 1 << 8; // Set (Buzzer OFF)
                }
            }
```

(Figure 8)

IERG3810_Buzzer.c > ...

```c
#include "stm32f10x.h"
//#include "main.h"

void IERG3810_Buzzer_Init(void){
    RCC ->APB2ENR |= 1 << 3;//1000(set IOPB to 1)

    GPIOB->CRH &= 0xFFFFFFF0;
    GPIOB->CRH |= 0x00000003;
    GPIOB->BRR = 1<<8; // reset (on)
}
```

(Figure 9)

IERG3810_KEY.c > ...

```c
#include "stm32f10x.h"
//#include "main.h"
void IERG3810_KEY_Init(void){
    // Enable clocks
    RCC->APB2ENR |= 1<<6; // GPIOE
    RCC->APB2ENR |= 1<<2; // GPIOA

    //key 0
    GPIOE->CRL &= 0xFFF0FFFF;//for bits 0-7, pin 2 is sixth(CNF7,6,5,...) clear sixth
    GPIOE->CRL |= 0x00080000;//assign 0x8 to sixth pos,0x3=0b1000,10 CNF 00 MODE
    GPIOE->BSRR|=1<<4;

    // KEY2 (PE2) input with pull-up
    GPIOE->CRL &= 0xFFFFF0FF;
    GPIOE->CRL |= 0x00000800;
    GPIOE->BSRR |= 1<<2;

    // KEY1 (PE3) input with pull-up
    GPIOE->CRL &= 0xFFFF0FFF;
    GPIOE->CRL |= 0x00008000;
    GPIOE->BSRR |= 1<<3;

    // KEY_UP (PA0) input with pull-down
    GPIOA->CRL &= 0xFFFFFFF0;
    GPIOA->CRL |= 0x00000008;
    GPIOA->BRR |= 1<<0;
}
```

(Figure 10)

IERG3810_LED.c > ...

```c
#include "stm32f10x.h"
//#include "main.h"

void IERG3810_LED_Init(void){
    RCC->APB2ENR |= 1<<3; // GPIOB
    RCC->APB2ENR |= 1<<6; // GPIOE
    // DS0 (PB5)
    GPIOB->CRL &= 0xFF0FFFFF;
    GPIOB->CRL |= 0x00300000;
    GPIOB->BSRR = 1<<5; // set (off)

    // DS1 (PE5)
    GPIOE->CRL &= 0xFF0FFFFF;
    GPIOE->CRL |= 0x00300000;
    GPIOE->BSRR = 1<<5; // set (off)

}
```

(Figure 11)

III. DISCUSSION

Question 1: What is open drain?

Answer 1: This experiment also demonstrates a basic wire-or logic behavior, where multiple input sources (buttons) can influence a single output (e.g., LED or buzzer). When using pull-up or pull-down resistors, pressing a button effectively pulls the line low, allowing the microcontroller to detect the change. This is similar to how wire-or logic allows multiple open-drain outputs to share a line and trigger a response when any one is active.

Question 2: Do you know how to configure a GPIO pin?

Answer 2: to configure a GPIO pin on an STM32 microcontroller by enable the port clock, set the pin mode and configuration using the CRL or CRH register, and control the pin output using BSRR or BRR.

Question 3: Which mode will you use for KEYs, LEDs and Buzzer?

Answer 3: For KEYs, you should use input mode with pull-up or pull-down depending on the circuit design. For LEDs and Buzzer, you should use output push-pull mode to drive them reliably.

IV. SUMMARY

<Summarize what have been found and studied in this experiment.>

In Lab 1, I learned how to configure GPIO pins on the STM32 board using both library functions and direct register settings. I started by setting up LEDs and buttons, then added delay control and input-based LED responses. Later, I implemented toggle functions for LEDs and the buzzer using multiple buttons. Finally, I modularized the code by creating custom initialization functions and organizing them into separate .c and .h files under a “Board” folder. This improved code readability and made the project easier to manage and extend.

V. DIVISION OF WORK

<Tabulate the division of work between group members>

N/A

VI. REFERENCES

<List out the source of the materials quoted/referenced, if any>

**Remember to attach the T.A.-signed & dated raw data sheets.

Note: Please follow the formatting requirements:

Section title: 12pts, Times New Roman, Bold

Text body: 11pts Times New Roman

Page margins 1 inch from top, bottom, left and right

Line Spacing: At least 14pts or Single